

# Tutorial 1: Printf and composing functions

Matteo Fasiolo

## Part A: printf

The standard-library function `printf` displays **formatted output**.

It is declared in the header file `<stdio.h>`, so our program needs:

```
#include <stdio.h>
```

In the lab, we have used `printf` to produce command-line output.

`printf` can do much more than simply print fixed text messages.

The general form is:

```
printf("FORMAT_STRING", VALUE1, VALUE2, ...);
```

The format string can contain **format specifiers**.

printf replaces each format specifier with the corresponding supplied value.

For example:

```
printf("My name is %s %s.\n", "Matteo", "Fasiolo");
```

This prints:

```
My name is Matteo Fasiolo.
```

Some useful format specifiers are:

- `%s`: a string
- `%f`: a decimal floating-point number
- `%.5f`: a floating-point number with five decimal digits
- `%d`: an integer
- `%c`: a single character

Backslash introduces an **escape sequence**, which represents a special character. For example:

- `\n`: start a new line

[Read this manual](#) for a list of all possible “format specifiers” and simple examples.

But this is pretty much all we need for this course!

# Exercise 1

Continue working in the `Lab_main.c` program from `Week_1_A`. Add further `printf` statements so that it prints:

1. Your student ID, such as `ab1234`, as a string.
2. Your student ID, treating its numerical part (`1234`) as an integer.
3. The result of  $1/3$  with three digits after the decimal point.
  - If it does not work, try to think about what is going on.

4. Add further `printf` statements to the same file to print this triangle:



A right-angled triangle is shown, composed of backslashes (\) and vertical bars (|). The left side is a vertical line of 5 bars. The top-right side is a diagonal line of 5 backslashes. The bottom side is a horizontal line of 4 dashes (-). The triangle is formed by the intersection of these lines.

**Hint:** inside a C string, `\\` represents one backslash. Therefore:

```
printf("\\");
```

prints \.

This is similar to a question from a past exam.

## Part B: Function composition and the chain rule

A **composition** uses the output of one function as the input of another.

If

$$u = h(x), \quad g(x) = k(u) = k(h(x)),$$

evaluate the functions **from the inside out**: first calculate  $h(x)$ , then pass its result to  $k$ .

The **chain rule** gives the derivative of the composition:

$$g'(x) = \frac{dg}{dx} = k'(h(x)) h'(x).$$

In words:

1. differentiate the outer function
2. evaluate it at the output of the inner function
3. multiply by the derivative of the inner function.

The nested function calls in this exercise follow this pattern.

Open `composite_funcs.c` and start with the family of functions

$$f(a, b, x) = \sin(ax + b).$$

1. Write a C function with the following header:

```
double f(double a, double b, double x)
```

The file already includes `<math.h>`, so you can use `sin(...)`.

**NOTE:** to compile the program on the terminal, use:

```
gcc composite_funcs.c -o composite_funcs.out -lm
```

The `-lm` option includes the maths library and must come at the end.

**Do not use the Run or Debug buttons in VScode**, which the `-lm` option when compiling.

2. Compose two members of this family by defining

$$g(a_{\text{out}}, b_{\text{out}}, a_{\text{in}}, b_{\text{in}}, x) = f(a_{\text{out}}, b_{\text{out}}, f(a_{\text{in}}, b_{\text{in}}, x)).$$

Write a C function with the following header:

```
double g(  
    double outer_a, double outer_b,  
    double inner_a, double inner_b, double x)
```

First evaluate the **inner** call,  $f(a_{\text{in}}, b_{\text{in}}, x)$ ; its returned value becomes the third argument of the **outer** call.

3. In main, set

```
double outer_a = 1.0;  
double outer_b = 0.5;  
double inner_a = 0.5;  
double inner_b = 0.1;  
double x = 1.5;
```

Pass these values to `g` and print the result with a clear label.

Your output should begin:

```
g(1.5) = 0.949388
```

4. Write a helper function `double f_derivative(double a, double b, double x)` that returns the derivative of  $f(a, b, x)$  with respect to  $x$ :

$$\frac{\partial f}{\partial x}(a, b, x) = a \cos(ax + b).$$

5. Use `f` and `f_derivative` to compute

```
double g_derivative(  
    double outer_a, double outer_b,  
    double inner_a, double inner_b, double x)
```

To do this, note that the chain rule implies

$$\frac{\partial g}{\partial x} = f_x(a_{\text{out}}, b_{\text{out}}, u) f_x(a_{\text{in}}, b_{\text{in}}, x),$$

where  $f_x$  denotes `f_derivative` and

$$u = f(a_{\text{in}}, b_{\text{in}}, x).$$

Call `g_derivative` with the values already defined in `main` and print its result. You should obtain:

```
g'(1.5) = 0.103653
```

# Background: The chain rule in AI and ML

Why does the chain rule matter?

Many methods in AI and machine learning are built by **composing simple functions**:

data  $\longrightarrow$  layer 1  $\longrightarrow$  layer 2  $\longrightarrow \dots \longrightarrow$  layer N  $\longrightarrow$  prediction  $\longrightarrow$  error.

Layer 1, ... layer N are functions that transform the data.

The input of layer  $i$  is the output of layer  $i-1$  and its output is the input of layer  $i + 1$ .

This is a composition of functions.

A neural network may contain millions of parameters. Training it means finding how each parameter affects the final error.

**Backpropagation** works backwards through the network, repeatedly applying the chain rule to compute these derivatives.

Libraries (such as PyTorch) do this automatically, but the mathematical idea underneath is the same one used in this exercise.

Your function  $g$  is a tiny two-layer model: first evaluate the inner function, then pass its result to the outer function and propagate the derivative back through both.